

Proposal for biolineage

Additional information regarding documentation for this project can be found in `overview.md`

What is this document?

This proposal describes the unified data model, ingestion pipeline, and normalization strategy used by **biolineage**, the successor to **norm.app**. It consolidates the work performed during the migration from SQLite to PostgreSQL, the redesign of the relationship graph, and the formalization of place, event, and lineage semantics.

The goal of this document is to provide Norm and Charles with:

- a clear explanation of how raw genealogical data is transformed into a structured, queryable, and historically accurate graph
- a rationale for each subsystem (relationships, events, places, gender inference, tree rendering, API integration)
- a record of all normalization decisions, including ambiguous or historically complex cases
- a stable foundation for future ingestion, editing, and expansion of the dataset

This proposal is supported by the following artifacts in `docs/`:

- **places-issues.md** — narrative explanation of locality normalization and all ambiguous cases
- **places.xlsx** — the full dataset of extracted place rows, including type classifications and discussion flags
- **transform.js** — the operational transformation pipeline used during ingestion
- **20260802-series documents** — detailed subsystem specifications (schema overview, event ingestion, relationship model, etc.)

Together, these documents define the complete data model and processing methodology for biolineage.

Technology Stack

The biolineage platform is intentionally minimal, predictable, and deployable in environments ranging from bare-metal Linux to Dockerized cloud instances. Each component was chosen for stability, performance, and long-term maintainability.

Nginx (static file delivery + reverse proxy)

Nginx serves two roles:

1. **Static file delivery** It provides fast, zero-overhead delivery of assets such as the lineage tree visualizations, client-side scripts, and documentation pages.
2. **Reverse proxy** It forwards API requests to the Node.js Fastify server, enabling:
 - TLS termination
 - clean routing
 - separation of static vs dynamic workloads
 - future load-balancing or multi-instance deployments

Nginx was chosen because it is stable, widely supported, and ideal for high-throughput static content.

Node.js (application runtime behind reverse proxy)

Node.js powers the ingestion pipeline, API layer, and lineage computation engine. It was selected because:

- its event-driven model is ideal for I/O-heavy workloads (database queries, ingestion streams)
- it integrates cleanly with Fastify
- it allows the entire platform (ingestion, normalization, API) to be written in one language
- it is easy to deploy in Docker or bare-metal environments

Node.js is the execution environment for all transformation logic, including `transform.js`, relationship inference, and place normalization.

Fastify (HTTP framework)

Fastify is used instead of Express because:

- it is significantly faster
- it has a strict schema-first philosophy
- it provides built-in validation, serialization, and plugin isolation
- it integrates cleanly with TypeScript (future direction)
- it is ideal for building predictable APIs for lineage queries

Fastify is the backbone of the API endpoints used by the entity tree renderer, event lookup, and place resolution.

PostgreSQL (primary data store)

PostgreSQL is the authoritative database for biolineage. It was chosen because:

- it supports complex relational queries needed for lineage traversal
- it handles recursive CTEs efficiently (ideal for ancestor/descendant graphs)
- it provides strong typing for places, events, and relationships
- it is stable, widely supported, and ideal for long-term archival data
- it integrates well with Node.js via `pg`

PostgreSQL replaces SQLite as the primary store, though SQLite remains useful for comparison and validation during ingestion.

Redis (cache and ephemeral store)

Redis is used as the in-memory cache layer for biolineage. It was chosen because:

- it provides extremely fast key/value access for repeated lineage tree queries
- it reduces PostgreSQL load during ancestor/descendant traversal
- it supports caching of normalized place lookups
- it enables future session storage when OIDC SSO is implemented
- it integrates cleanly with Node.js through lightweight client libraries
- it is stable, widely supported, and ideal for ephemeral or high-frequency data

Redis does not store authoritative data. It accelerates computation and improves responsiveness across the API.

Docker (deployment target)

Although the current environment runs directly on the host, Docker is the intended deployment target for:

- reproducible installs
- isolated environments
- versioned database containers
- easy migration between servers
- future multi-instance scaling

The platform is designed so that each component (Nginx, Node.js, PostgreSQL) can be containerized independently.

Other technologies (not emphasized)

Some components are intentionally omitted from the proposal because they are implementation details rather than architectural choices:

- `dotenv` — environment variable loading
- `nodemon` — development auto-restart
- `sharp` — image processing
- `argon2` — password hashing
- `better-sqlite3` — ingestion comparison

These are part of the implementation but not core architectural decisions.

Here is the **final section** you want for `proposal.md`: a clean, structured set of descriptions for every file in `docs/`. This section belongs at the end of the proposal, after all subsystem explanations, and serves as a reference index for Norm and Charles.

I'll write it in a way that fits the tone of the proposal packet and aligns with the architecture you've already established.

Appendix: Documentation Index

This section provides a description of each file in the `docs/` directory. These documents collectively define the data model, ingestion pipeline, normalization methodology, and API behavior of **biolineage**. They are intended to be read alongside the main proposal to provide deeper technical detail where needed.

20260802-schema-overview.md

A high-level description of the PostgreSQL schema used by biolineage. It explains:

- core tables (entities, events, relationships, places)
- foreign-key structure
- UUID strategy
- how lineage, events, and places interrelate
- rationale for each table's shape

This document is the authoritative reference for the database model.

20260802-sqlite-to-postgresql-tree-comparison.md

A comparison between the original SQLite lineage tree logic and the redesigned PostgreSQL implementation. It covers:

- differences in recursion strategy
- improvements in ancestor/descendant traversal
- correctness issues found in the SQLite version
- validation tests comparing both outputs

This document demonstrates why PostgreSQL is the correct long-term home for lineage computation.

20260802-relationship-model-rationale.md

A detailed explanation of the relationship graph model. It includes:

- parent/child semantics
- spouse and partner relationships
- inferred relationships
- how ambiguous or conflicting data is resolved
- why the graph-first model is superior to GEDCOM-style hierarchical trees

This document is essential for understanding how biolineage computes lineage.

20260802-event-ingestion.md

A description of how raw event data (birth, death, burial, marriage, immigration, etc.) is transformed into normalized PostgreSQL event rows. It explains:

- parsing rules
- event typing
- date normalization
- place linkage
- handling of incomplete or narrative event descriptions

This document defines the ingestion pipeline for all event data.

20260802-place-normalization.md

The formal specification for place normalization. It covers:

- how raw BirthCountry/BurialCountry/etc fields are parsed
- how ogName / ogCountry / ogRegion / ogCity are interpreted
- how UUIDs are assigned
- how misspellings, counties, parishes, and historical names are resolved
- how ambiguous cases are flagged for human review

This document is the methodology behind `places-issues.md` and `places.xlsx`.

20260802-sex-inference.md

A description of the sex inference subsystem. It explains:

- how sex is inferred from relationships
- how ambiguous cases are handled
- how conflicting data is resolved
- how sex is stored and updated in the database

This document defines the logic used to infer sex when Norm's dataset does not explicitly provide it.

20260802-tree-function-design.md

A technical description of the lineage tree rendering functions. It includes:

- ancestor traversal
- descendant traversal
- cycle detection
- ordering rules
- how the API returns tree structures to the frontend

This document defines the algorithms behind `/entity/:id/entity-tree`.

20260802-api-integration.md

A description of the API endpoints exposed by biolineage. It covers:

- request/response formats
- Fastify route structure
- serialization rules
- how the API integrates with the lineage tree renderer
- how the API will integrate with future frontend components

This document is the reference for all external integrations.

places-issues.md

A narrative document describing all ambiguous or problematic place entries extracted from Norm's dataset. It contains:

- every row marked **discussion** in `places.xlsx`
- detailed case-by-case analysis
- questions requiring human decision (Norm & Charles)
- historical locality issues
- misspellings, county/city confusion, international transliteration problems

This document is the human review log for place normalization.

places.xlsx

The full dataset of extracted place rows. It includes:

- raw values from Norm's BirthCountry/BurialCountry/etc fields
- normalized fields (ogName, ogCountry, ogRegion, ogCity)
- UUIDs
- type classifications (normalized, fixable, discussion, ambiguous)
- links to **places-issues.md** for all discussion rows

This spreadsheet is the authoritative dataset for place normalization.

transform.js

The operational ingestion pipeline. It contains:

- parsing logic
- normalization rules
- the **placeFixes** array
- UUID resolution
- deterministic output functions
- the glue between raw Norm data and the PostgreSQL schema

This file is the executable implementation of the methodology described in the proposal.

Additional immutable Appendix information can be found in **appendix.md**

overview.md

Documentation Overview for biolineage

This document provides an overview of the biolineage documentation set. It explains the purpose of each file in the docs/ directory, the milestone-based structure, and how the appendix fits into the overall documentation system.

Future Work

Proposals for upcoming architectural changes can be found in the proposals/ directory. Proposals represent planned architectural changes. They are not milestones and do not describe completed work.

1. Documentation Structure

The documentation in the docs/ directory is divided into two categories:

1. milestone documents

2. immutable appendix information

Milestone documents describe architectural decisions, ingestion logic, normalization rules, and API behavior at specific points in time. They are versioned using date prefixes.

The appendix contains permanent project-level information that does not change across milestones.

2. Milestone Documents

Milestone documents use the format:

`YYYYMMDD-topic.md`

Each milestone captures the state of a subsystem at a specific point in time. The current milestone set includes:

20260802-relationship-model-rationale.md

Explains the graph-first relationship model, including parent/child semantics, spouse and partner edges, inferred relationships, ambiguity handling, and why graph traversal is superior to hierarchical GEDCOM structures.

20260802-event-ingestion.md

Describes how raw event data is transformed into normalized PostgreSQL rows. Covers parsing rules, event typing, date normalization, place linkage, and handling of incomplete or narrative event descriptions.

20260802-place-normalization.md

Defines the formal methodology for place normalization. Explains how raw locality fields are parsed, how `ogName` / `ogCountry` / `ogRegion` / `ogCity` are interpreted, how UUIDs are assigned, how misspellings and historical names are resolved, and how ambiguous cases are flagged.

20260802-sex-inference.md

Describes the sex inference subsystem. Covers inference from relationships, narrative context, ambiguous cases, conflict resolution, and how sex is stored and updated in the database.

20260802-tree-function-design.md

Explains the lineage tree rendering functions. Covers ancestor traversal, descendant traversal, cycle detection, ordering rules, sibling derivation, spouse handling, and the structure returned by the entity tree API.

20260802-api-integration.md

Documents the API endpoints exposed by biolineage. Covers Fastify route structure, request and response formats, serialization rules, lineage tree integration, and future frontend compatibility.

3. Immutable Appendix

The appendix.md file contains permanent project-level information, including:

- project ownership
- copyright assignment
- attribution
- AI involvement
- geographical data sources
- tandem geography project schema
- static site information
- long-term hosting commitment

This information does not change across milestones and is not versioned by date.

4. How Milestones Work

Milestones represent stable snapshots of architectural decisions. New milestones are created when:

- ingestion logic changes
- normalization rules evolve
- new subsystems are added
- schema changes occur
- API behavior is updated

Milestones are not edited after creation. Future changes are documented in new dated files.

5. Relationship to the Codebase

The documentation corresponds directly to the implementation:

- ingestion logic in transform.js
- relationship and event tables in PostgreSQL
- place normalization rules in places.xlsx and placeFixes
- lineage tree functions in the entity tree endpoint
- API routes in Fastify plugins

- geography schema in db/pg-definitions.js

Each milestone document maps to a specific subsystem in the codebase.

6. Navigation

For a high-level understanding, read:

- overview.md
- appendix.md

For subsystem details, read the milestone documents in chronological order.

7. Summary

The docs/ directory contains a complete, dated, and structured record of the biolineage architecture. Milestones capture technical decisions at specific points in time, while the appendix provides permanent project metadata. This overview serves as the entry point for understanding the documentation set.

20260802-schema-overview.md

PostgreSQL Schema Overview for biolineage

This document provides a high-level description of the PostgreSQL schema used by **biolineage**, including the core tables, foreign-key structure, UUID strategy, and the rationale behind each component. It is the authoritative reference for how lineage, events, relationships, and places interrelate within the system.

1. Guiding Principles of the Schema

The schema is designed around several core principles:

- **Graph-first lineage modeling** — relationships are edges, entities are nodes.
- **Event-centric historical modeling** — births, deaths, burials, marriages, migrations, etc. are first-class entities.
- **Normalized place representation** — every locality is a structured, deduplicated, UUID-identified row.
- **Deterministic ingestion** — raw data from Norm's dataset is transformed into stable, predictable rows.
- **Historical accuracy** — counties, parishes, towns, and historical names are preserved when meaningful.

- **Future extensibility** — species, clades, and sovereign entities can be added without schema changes.

These principles shape every table described below.

2. Core Tables

The schema consists of four foundational tables:

- **entities**
- **events**
- **relationships**
- **places**

Each table is designed to be minimal, strongly typed, and linked through UUIDs.

3. UUID Strategy

All primary keys in biolineage are **UUIDv4**. This ensures:

- global uniqueness
- safe merging of external datasets
- deterministic ingestion (UUIDs assigned before insertion)
- no reliance on sequential IDs
- compatibility with distributed or Dockerized deployments

UUIDs are generated during ingestion and stored in the transformation pipeline (`transform.js`).

4. entities Table

The **entities** table represents individuals in the lineage graph.

Fields include:

- `uuid` — primary key
- `given_name`
- `family_name`
- `gender` — inferred or explicit
- `notes` — narrative text
- `created_at` / `updated_at`

Rationale:

The entities table is intentionally minimal. All temporal and spatial information is stored in **events**, and all relational information is stored in **relationships**. This keeps the entity clean and avoids duplication.

Perfect — now that all the incorrect *person/people* usages are fixed, what you need is a **clear, authoritative section** in the docs that explains *why Biolineage uses “entity” instead of Norm’s original “person” table*. This section should justify the terminology shift, anchor it in real domain needs, and prevent future confusion for contributors.

Below is a fully-formed documentation section you can drop directly into your .md files. It’s structured, semantically precise, and grounded in the actual architectural motivations you described.

Entity Model Rationale

Biolineage models **entities** rather than “persons,” diverging intentionally from the original **person** table in Norm’s early schema. This change reflects the broader scope of lineage modeling and corrects several semantic and architectural limitations inherent in human-only terminology.

Why “entity” is the correct abstraction

- **Semantic neutrality** — “Person” implies a biological human. Biolineage needs a term that applies equally to humans, animals, organizations, artifacts, and other lineage-bearing nodes. “Entity” is the only term that scales cleanly across domains.
- **Multi-species support** — Modern lineage systems must handle horses, dogs, livestock, lab organisms, and historical non-human subjects. The original “person” table cannot represent these without semantic distortion. “Entity” avoids anthropocentric bias.
- **Instigator model alignment** — Biolineage’s ingestion pipeline treats the source record (GEDCOM individual, horse registry entry, manuscript provenance record, etc.) as the *instigator* of an entity. Using “entity” keeps the conceptual boundary clean: the instigator is the record; the entity is the graph node created from it.
- **Better ontology for lineage graphs** — Lineage graphs often include non-person nodes: farms, stables, breeding programs, labs, estates, and events. “Entity” allows these to coexist naturally without special-case tables.
- **Future-proofing** — As Biolineage expands into historical sovereignty, manuscript lineage, or biological sample tracking, “entity” remains valid. “Person” would require repeated schema migrations or awkward renaming.

Relationship to Norm’s original person table

Norm’s early schema used a **person** table because the initial scope was human genealogy. Once horse records and other non-human datasets were introduced, the limitations became obvious:

- Horse registries treat animals as first-class lineage subjects.
- Breed associations track ancestry exactly like human genealogical systems.
- Many datasets mix humans and non-humans (e.g., breeder → horse → offspring).

Rather than creating parallel tables (**person**, **horse**, **animal**, etc.), Biolineage unifies them under **entity**, with type metadata determining the specific category.

Practical consequences for contributors

- When referring to the **graph node**, always use **entity**.
- When referring to a **biological human**, use **person**.
- When referring to a **GEDCOM record**, use **individual**.
- When referring to **source provenance**, use **instigator**.
- When referring to **relationship roles**, use **subject/object** or domain-specific terms (parent, child, sire, dam).

This keeps the documentation consistent, avoids anthropocentric assumptions, and ensures the model remains extensible.

5. events Table

Events represent historical occurrences tied to entities.

Fields include:

- **uuid** — primary key
- **entity_uuid** — FK → entities
- **event_type** — birth, death, burial, marriage, immigration, etc.
- **date** — normalized date or partial date
- **place_uuid** — FK → places
- **description** — narrative text
- **source** — optional provenance

Rationale:

Events allow biolineage to model history in a structured way. Birth, death, and burial are not attributes of an entity — they are **events**. This enables:

- multiple events per entity
- partial dates
- ambiguous or narrative descriptions
- future event types (military service, census, migration, etc.)

6. relationships Table

Relationships define edges in the lineage graph.

Fields include:

- `uuid` — primary key
- `entity_a_uuid` — FK → entities
- `entity_b_uuid` — FK → entities
- `relationship_type` — parent, child, spouse, partner, sibling, etc.
- `inferred` — boolean
- `notes` — optional narrative

Rationale:

The relationship model is **graph-first**. Every relationship is directional and explicit. Parent/child relationships are stored as edges, not embedded in the entity record.

This enables:

- recursive ancestor/descendant traversal
 - cycle detection
 - multi-parent or blended families
 - inferred relationships from event data
 - future expansion to species/clade lineage
-

7. places Table

Places represent normalized localities.

Fields include:

- `uuid` — primary key
- `name` — canonical name
- `country` — ISO-3166-1 alpha-2
- `region` — state/province/county/parish
- `city` — town/hamlet/locality
- `type` — city, county, parish, cemetery, hospital, region, discussion, etc.
- `notes` — narrative or historical context

Rationale:

Place normalization is one of the most complex parts of the ingestion pipeline. Raw fields from Norm's dataset (`BirthCountry`, `BurialCountry`, etc.) are parsed into:

- `ogName`

- `ogCountry`
- `ogRegion`
- `ogCity`

These are then normalized into structured place rows using:

- `places.xlsx` (dataset)
- `places-issues.md` (discussion cases)
- `transform.js` (operational logic)

Every place is assigned a UUID, ensuring deterministic linkage from events.

8. Foreign-Key Structure

The schema uses a simple, predictable FK structure:

- **events** → **entities**
- **events** → **places**
- **relationships** → **entities (twice)**
- **places** → **none** (places are standalone)

This structure ensures:

- entities are the root nodes
 - events attach temporal/spatial meaning
 - relationships attach graph edges
 - places attach locality meaning
 - no circular dependencies
 - clean deletion rules (events and relationships cascade)
-

9. How the Tables Interrelate**

The schema forms a three-layer model:

Layer 1: Entities

The nodes of the lineage graph.

Layer 2: Events

Historical occurrences tied to entities.

Layer 3: Relationships

Edges connecting entities.

Layer 4: Places

Normalized localities referenced by events.

This creates a unified model:

- entities → events → places
- entities → relationships → entities

Everything else (tree rendering, gender inference, normalization) builds on this foundation.

10. Rationale for the Schema Shape

The schema is intentionally:

- **normalized** — no duplication of place or event data
- **graph-oriented** — relationships are edges
- **event-centric** — history is modeled as events
- **place-aware** — localities are structured and deduplicated
- **extensible** — species, clades, and sovereign entities can be added without redesign
- **deterministic** — ingestion produces stable UUIDs and consistent rows
- **historically accurate** — counties, parishes, and historic names are preserved

This design supports both modern genealogy and historical lineage modeling.

20260802-relationship-model-rationale.md

Relationship Model Rationale for biolineage

This document explains the rationale behind the relationship graph model used by biolineage. It describes how parent/child semantics are represented, how spouse and partner relationships are modeled, how inferred relationships are generated, how ambiguous or conflicting data is resolved, and why a graph-first model is superior to GEDCOM-style hierarchical trees.

The relationship model is one of the core architectural foundations of biolineage and directly informs lineage traversal, gender inference, event aggregation, and tree rendering.

1. Overview

The relationship model in biolineage is designed around a simple principle:

Entities are nodes. Relationships are edges.

This graph-first approach allows the system to represent complex family structures, blended families, inferred parentage, historical ambiguity, and multi-generational lineage without the constraints of hierarchical tree formats such as GEDCOM.

The model supports:

- explicit parent/child edges
 - spouse and partner edges
 - inferred edges derived from events
 - ambiguous or conflicting parentage
 - cycle detection
 - multi-parent families
 - future expansion to species, clades, and sovereign entities
-

2. Parent/Child Semantics

Parent/child relationships form the backbone of the lineage graph. Each relationship is represented as a directional edge:

- parent $\rightarrow$ child
- child $\rightarrow$ parent

The `relationship_type` field stores the semantic meaning of the edge. Parent/child edges are always explicit and directional, allowing:

- ancestor traversal
- descendant traversal
- generation-level ordering
- cycle detection
- gender inference
- event linkage (birth, baptism, adoption)

Parent/child edges are never inferred from spouse relationships or sibling relationships. They must originate from:

- explicit data in Norm's dataset
 - inferred parentage from birth events
 - manual correction during ingestion
 - future editorial input
-

3. Spouse and Partner Relationships

Spouse and partner relationships are modeled as undirected edges represented by two directional rows:

- A $\rightarrow$ B (spouse)

- $B \rightarrow A$ (spouse)

This ensures:

- symmetric traversal
- correct rendering in entity trees
- ability to detect blended families
- ability to infer shared parentage
- correct handling of multiple marriages or partnerships

Spouse edges are never treated as ancestor or descendant edges. They exist solely to represent marital or partnership connections.

Partner relationships (non-marital) use the same structure but a different `relationship_type`.

4. Inferred Relationships

Inferred relationships are generated when event data strongly implies a parent/child connection. Examples include:

- a birth event with a listed mother or father
- a baptism event with godparents or guardians
- a burial event referencing next-of-kin
- a marriage event referencing parents of the bride or groom

Inferred edges are marked with `inferred = true`.

This allows the system to:

- include parentage even when Norm's dataset lacks explicit relationship rows
- maintain correctness in lineage traversal
- support gender inference
- fill gaps in historical data

Inferred edges never override explicit edges. They supplement them.

5. Ambiguous or Conflicting Data

Historical datasets often contain ambiguity:

- multiple possible fathers
- conflicting birth records
- missing mothers
- blended families
- adoption vs biological parentage
- step-parents

- guardians
- narrative-only descriptions

The relationship model resolves ambiguity using deterministic rules:

1. Explicit parentage always wins.
2. Inferred parentage is included only when unambiguous.
3. Conflicting inferred parents are flagged for review.
4. Multiple parents are allowed.
5. Cycles are detected and prevented.
6. Narrative-only relationships are stored as notes.
7. Ambiguous edges are stored but marked `ambiguous = true` (future extension).

This ensures the lineage graph remains correct, complete, and stable.

6. Why Graph-First Is Superior to GEDCOM Hierarchies

GEDCOM represents families as hierarchical trees:

- one family unit
- two parents
- multiple children
- rigid structure
- limited support for blended families
- limited support for historical ambiguity
- no inferred relationships
- no multi-parent families
- no cycle detection
- no graph traversal semantics

The graph-first model used by biolineage solves all of these limitations.

Advantages of the graph model

- supports any number of parents
- supports blended families
- supports historical ambiguity
- supports inferred relationships
- supports cycle detection
- supports deep recursion
- supports species/clade lineage
- supports sovereign entity lineage
- supports event-driven parentage inference
- supports future editorial corrections
- supports multi-generational traversal without structural constraints

The graph model is the correct representation for real-world lineage data.

7. Relationship Table Structure

The relationships table contains:

- uuid
- entity_a_uuid
- entity_b_uuid
- relationship_type
- inferred
- notes

This minimal structure allows:

- directional edges
- symmetric spouse edges
- multi-parent families
- inferred edges
- ambiguous edges
- future expansion
- clean traversal logic
- deterministic ingestion

The simplicity of the table is intentional. All complexity lives in traversal logic, not in the schema.

8. How the Relationship Model Supports Lineage Computation

The graph model enables:

- ancestor traversal
- descendant traversal
- cycle detection
- generation-level ordering
- gender inference
- event aggregation
- tree rendering
- place-linked event traversal
- future clade/species lineage modeling

The entity tree endpoint relies entirely on this model.

9. Summary

The relationship model in biolineage is designed to be:

- simple
- expressive
- correct
- extensible
- historically accurate
- graph-first
- event-aware
- ambiguity-tolerant

It is superior to hierarchical formats and is the foundation of all lineage computation in the platform.

20260802-event-ingestion.md

Event Ingestion Pipeline for biolineage

This document describes how raw event data (birth, death, immigration, and narrative historical events) is transformed into normalized PostgreSQL event rows. It explains the parsing rules, event typing, date normalization, place linkage, and handling of incomplete or narrative event descriptions. The event ingestion pipeline is a core part of the biolineage data model and ensures that historical information is represented consistently and accurately.

1. Overview

Events are first-class entities in biolineage. Instead of storing birth or death information directly on the entity record, each occurrence is represented as an event row linked to the entity and to a normalized place. This event-centric model allows the system to represent:

- multiple events per entity
- partial or ambiguous dates
- narrative descriptions
- historical locality variations
- inferred relationships
- future event types without schema changes

Burial and marriage information are handled differently:

- Norm's dataset contains **burial places** but **no burial dates**, so burial events are not created.
- Norm's dataset contains **no marriage dates or marriage locality fields**, so marriage events are not created.

- Marriage is represented solely through relationship edges.
-

2. Raw Event Fields

Norm's dataset provides structured event information for:

- BirthCountry, BirthRegion, BirthCity
- DeathCountry, DeathRegion, DeathCity
- DeathDate
- BirthDate

It also provides:

- BurialCountry, BurialRegion, BurialCity (place only, no date)
- Narrative text fields
- Partial or ambiguous dates
- Historical locality names
- Misspellings or variants

There are **no marriage fields** and **no burial dates**.

3. Event Typing

Each event is assigned an event_type. The core types include:

- birth
- death
- immigration (only when detected in narrative text)
- miscellaneous (for narrative-only events)

Marriage and burial are intentionally excluded because the dataset does not contain dates for either.

Event typing is deterministic. For example:

- BirthCountry → birth
- DeathCountry → death

Narrative fields may produce additional event types when recognizable patterns are present.

4. Parsing Rules

The ingestion pipeline applies a series of parsing rules to convert raw fields into structured event rows.

4.1 Field extraction

Birth event:

- BirthCountry
- BirthRegion
- BirthCity
- BirthDate
- BirthNotes

Death event:

- DeathCountry
- DeathRegion
- DeathCity
- DeathDate
- DeathNotes

Burial fields are extracted for place normalization but do not produce an event.

Marriage fields do not exist in the dataset.

4.2 Null and placeholder handling

Common placeholder values include:

- unknown
- ?
- tbd
- narrative-only text
- empty strings

These are converted to null or moved into the description field.

4.3 Narrative extraction

Narrative fields are preserved in the description column. Examples:

- “Died young”
- “Some say born 1732”
- “Age 57.5 hours”

Narrative text is never discarded.

5. Date Normalization

Dates in Norm’s dataset appear in many formats:

- YYYY-MM-DD
- MM/DD/YYYY

- Month YYYY
- YYYY
- “circa 1819”
- “before 1900”
- “after 1750”
- “unknown”

The ingestion pipeline normalizes dates into one of three forms:

1. Full date
2. Partial date (year only or month/year)
3. Null date with narrative preserved

Examples:

Raw: “9/30/1819” → Normalized: 1819-09-30 Raw: “1732?” → Normalized: 1732 with uncertainty noted in description Raw: “circa 1819” → Normalized: null date, description preserved

Date normalization ensures consistent sorting and traversal.

6. Place Linkage

Every event is linked to a normalized place row via `place__uuid`.

Place normalization is handled by:

- `places.xlsx`
- `places-issues.md`
- `transform.js`
- the `placeFixes` array
- deterministic UUID assignment

The ingestion pipeline converts raw locality fields into:

- `ogName`
- `ogCountry`
- `ogRegion`
- `ogCity`

These are then resolved to a `place__uuid`.

Burial places follow the same normalization pipeline, but do not produce event rows.

7. Handling Incomplete or Narrative Event Descriptions

Many events contain incomplete or narrative-only information. Examples:

- “Died young”
- “Some say died 9/30/1819”
- “Age 57.5 hours”
- “Some say born 1732”

The ingestion pipeline handles these cases by:

1. Preserving narrative text in description
2. Linking to a normalized place when possible
3. Creating a miscellaneous event when no type can be inferred
4. Avoiding incorrect assumptions about event type or locality

Narrative text is always preserved because it may contain historically meaningful information.

8. Event Table Structure

The events table contains:

- uuid
- entity__uuid
- event__type
- date
- place__uuid
- description
- source

This minimal structure allows:

- multiple events per entity
- partial or null dates
- narrative descriptions
- linkage to normalized places
- future event types without schema changes

Burial locality is stored in the places table and linked to the entity through context, not through an event row.

Marriage is represented through relationship edges, not through events.

9. Ingestion Pipeline Flow

The ingestion pipeline follows this sequence:

1. Extract raw fields
2. Determine event type
3. Normalize date

4. Normalize place
5. Assign UUID
6. Construct event row
7. Insert into PostgreSQL
8. Log ambiguous cases for review

Burial locality is normalized but does not produce an event row. Marriage relationships are handled by the relationship ingestion pipeline, not the event pipeline.

10. Summary

The event ingestion pipeline in biolineage is designed to be:

- deterministic
- structured
- narrative-preserving
- place-aware
- historically accurate
- flexible
- extensible

Events are the backbone of historical modeling in the platform. Burial and marriage information are preserved through place normalization and relationship edges, respectively, but do not produce event rows due to the absence of structured date fields.

20260802-place-normalization.md

Place Normalization Methodology for biolineage

This document describes the formal methodology used to normalize place data in biolineage. It explains how raw BirthCountry/BurialCountry/etc fields are parsed, how ogName / ogCountry / ogRegion / ogCity are interpreted, how UUIDs are assigned, how misspellings and historical locality names are resolved, and how ambiguous cases are flagged for human review. This specification defines the logic behind places.xlsx, places-issues.md, and the ingestion pipeline implemented in transform.js.

1. Overview

Place normalization is one of the most complex components of the biolineage ingestion pipeline. Norm's dataset contains thousands of locality references in inconsistent formats, including:

- misspellings
- partial locality names
- counties treated as cities
- parishes treated as regions
- historical names
- narrative-only descriptions
- ambiguous or conflicting entries
- multiple variants of the same locality

The goal of place normalization is to convert these raw fields into structured, deduplicated, UUID-identified place rows in PostgreSQL.

2. Raw Place Fields

Norm’s dataset provides locality information through several structured fields:

- BirthCountry, BirthRegion, BirthCity
- DeathCountry, DeathRegion, DeathCity
- BurialCountry, BurialRegion, BurialCity

These fields may contain:

- country codes
- state or province abbreviations
- county names
- city names
- narrative text
- misspellings
- historical locality names
- placeholders such as “?”, “unknown”, “tbd”

The ingestion pipeline extracts these fields and converts them into normalized place rows.

3. Parsing Raw Fields into ogName / ogCountry / ogRegion / ogCity

Each raw locality field is parsed into four canonical components:

- ogName
- ogCountry
- ogRegion
- ogCity

These represent the original values exactly as they appear in the dataset, without correction. Examples:

Raw: “Sumter Cemetary, SC, USA” Parsed:

- ogName: Sumter Cemetary
- ogCountry: us
- ogRegion: SC
- ogCity: Sumter

Raw: “Leinfelden Stuttgart, BW, Germany” Parsed:

- ogName: null
- ogCountry: de
- ogRegion: BW
- ogCity: Leinfelden Stuttgart

Raw: “?” Parsed:

- ogName: ?
- ogCountry: null
- ogRegion: null
- ogCity: null

These parsed fields form the basis for normalization.

4. UUID Assignment

Every normalized place is assigned a UUID. UUIDs are deterministic and stored in places.xlsx. The ingestion pipeline uses the following rules:

1. If a placeFixes rule matches the raw fields, the UUID from the rule is used.
2. If a place already exists in places.xlsx, its UUID is reused.
3. If a new place is created, a new UUID is generated and added to places.xlsx.

This ensures stable linkage between events and places across multiple ingestion runs.

5. Normalization Rules

Normalization converts ogName / ogCountry / ogRegion / ogCity into structured place rows. The rules include:

5.1 Country normalization

Countries are normalized to ISO-3166-1 alpha-2 codes. Examples:

- USA → us
- United States → us
- Germany → de
- England → gb

Unknown or ambiguous countries remain null.

5.2 Region normalization

Regions may represent:

- states
- provinces
- counties
- parishes
- historical districts

Region normalization preserves historically meaningful names. Examples:

- SC → South Carolina
- BW → Baden-Württemberg
- Christ Church Parish → preserved as region
- Darlington District → preserved as historical region

5.3 City normalization

Cities may represent:

- towns
- hamlets
- communities
- villages
- neighborhoods
- districts

City normalization corrects misspellings when unambiguous. Examples:

- Gainesville → Gainesville
- Silver Spring → Silver Spring
- Jonhsonville → Johnsonville

Ambiguous city names are flagged for review.

5.4 Name normalization

ogName may represent:

- cemetery names
- hospital names
- churches
- farms
- estates
- narrative text
- partial locality names

Name normalization corrects misspellings when unambiguous. Examples:

- Cemetary → Cemetery
- Ole Johnsonville Cemetery → Old Johnsonville Cemetery

Narrative-only names are preserved but marked as type = discussion.

6. Misspellings, Variants, and Historical Names

The normalization pipeline handles:

- spelling variants
- historical locality names
- county vs city confusion
- parish vs region confusion
- multiple variants of the same cemetery
- multiple variants of the same town
- international transliteration issues

Examples:

- Escambria → Escambia
- Musberg Stuttgart → Musberg (district of Leinfelden-Echterdingen)
- Poyngyang → Pyongyang
- Hamung → Hamhung

Historical names are preserved when meaningful. Examples:

- Dorsetshire → Dorset
 - Darlington District → preserved as historical region
-

7. Ambiguous Cases and Human Review

Some localities cannot be normalized automatically. These are flagged as type = discussion in places.xlsx and described in detail in places-issues.md.

Examples include:

- New York vs Orange (New York)
- Texas vs Corsicana (Texas)
- Leinfelden Stuttgart vs Musberg Stuttgart
- Hamung vs Hamongji vs Jungpyongkun
- Silver Valley (multiple states)
- Escambia vs Escambria
- Orangetown vs Tappan
- Anderson County Hospital vs Anderson County Cemetery

Ambiguous cases require human decision from Norm and Charles.

8. Place Types

Each normalized place is assigned a type. Examples:

- city
- county
- parish
- region
- cemetery
- hospital
- discussion
- ambiguous

Type classification is stored in `places.xlsx` and used during ingestion and event linkage.

9. Normalization Pipeline Flow

The normalization pipeline follows this sequence:

1. Extract raw locality fields
2. Parse into `ogName` / `ogCountry` / `ogRegion` / `ogCity`
3. Apply `placeFixes` rules
4. Normalize country, region, city, and name
5. Assign UUID
6. Classify type
7. Insert or update `places.xlsx`
8. Flag ambiguous cases for human review
9. Document issues in `places-issues.md`

This deterministic flow ensures stable output across ingestion runs.

10. Summary

The place normalization methodology in `biolineage` is designed to be:

- deterministic
- structured
- historically accurate
- narrative-preserving
- ambiguity-aware
- extensible
- consistent across ingestion runs

`places.xlsx` contains the authoritative dataset of normalized places. `places-issues.md` contains the human review log for ambiguous cases. `transform.js`

contains the operational implementation of this methodology.

20260802-sex-inference.md

Sex Inference Methodology for biolineage

This document describes the sex inference subsystem used by biolineage. It explains how sex is inferred from relationships, how ambiguous cases are handled, how conflicting data is resolved, and how sex is stored and updated in the database. The sex inference logic is used only when Norm's dataset does not explicitly provide sex information.

1. Overview

Norm's dataset contains many individuals without explicit sex fields. To support lineage traversal, relationship modeling, and event interpretation, biolineage infers sex when reliable structural evidence exists.

Sex inference is based solely on:

- explicit parent/child relationships
- spouse or partner relationships
- event context (e.g., "mother", "father" in narrative text)
- deterministic rules that avoid assumptions

Sex is never inferred from names, cultural stereotypes, or historical generalizations.

2. Sources of Sex Information

Sex may originate from three sources:

1. Explicit sex provided in Norm's dataset
2. Inferred sex from relationship structure
3. Inferred sex from event context

Explicit sex always takes precedence.

3. Inferring Sex from Parent/Child Relationships

Parent/child edges provide the strongest structural evidence for sex inference.

Examples:

- If an entity is explicitly listed as the mother of a child, sex is inferred as female.
- If an entity is explicitly listed as the father of a child, sex is inferred as male.

Norm’s dataset does not contain explicit “mother” or “father” labels, but parentage can be inferred from event context or relationship structure.

When two parents are listed for a child:

- If one parent already has explicit sex, the other parent may be inferred when the relationship type indicates a spouse or partner.
- If both parents have explicit sex, no inference is needed.
- If neither parent has explicit sex, no inference is made.

Parent/child relationships never imply sex unless additional context exists.

4. Inferring Sex from Spouse or Partner Relationships

Spouse or partner relationships do not automatically imply sex. However, they may contribute to inference when combined with parent/child edges.

Examples:

- If two individuals are spouses and one is explicitly the mother of a child, the other may be inferred as the father when the relationship structure supports it.
- If two individuals are spouses and one has explicit sex, the other is not inferred unless parentage context exists.

Spouse relationships alone are not sufficient to infer sex.

5. Inferring Sex from Event Context

Narrative event descriptions occasionally contain sex-specific terms such as:

- mother
- father
- widow
- widower

When these terms appear in narrative text, they may be used to infer sex.

Examples:

- “Widow of John Smith” → inferred female
- “Widower of Mary Jones” → inferred male

Narrative context is used only when the term is unambiguous.

6. Ambiguous Cases

Many individuals in Norm’s dataset lack sufficient information for sex inference. Ambiguous cases include:

- individuals with no relationships
- individuals with only sibling relationships
- individuals with only spouse relationships and no children
- individuals with narrative-only event descriptions
- individuals with conflicting or unclear parentage

Ambiguous cases remain with sex = null.

No assumptions are made based on:

- given names
- family names
- cultural naming patterns
- historical norms
- statistical likelihood

Sex is inferred only when structural evidence exists.

7. Conflicting Data

Conflicting data may occur when:

- narrative text contradicts relationship structure
- multiple inferred relationships suggest different sexes
- historical records contain inconsistent information

Conflict resolution rules:

1. Explicit sex always wins.
2. Inferred sex from parent/child edges overrides narrative inference.
3. Narrative inference overrides spouse-only inference.
4. If two inference sources conflict, sex remains null.

This ensures correctness and avoids incorrect assumptions.

8. How Sex Is Stored and Updated

Sex is stored in the entities table as:

- male
- female

- null (unknown or ambiguous)

During ingestion:

1. Explicit sex is applied first.
2. Inferred sex is applied only when unambiguous.
3. Conflicting inferred sex results in null.
4. Sex is never overwritten once set explicitly.
5. Inferred sex may be overwritten by explicit sex if provided later.

This ensures deterministic and stable sex assignment.

9. Interaction with Other Subsystems

Sex inference supports:

- relationship traversal
- ancestor and descendant tree rendering
- event interpretation
- spouse and partner modeling
- future editorial corrections

Sex is not required for lineage computation, but it improves clarity and correctness in tree rendering and event context.

10. Summary

The sex inference subsystem in biolineage is designed to be:

- deterministic
- conservative
- structurally grounded
- ambiguity-aware
- free of cultural or naming assumptions
- correct across ingestion runs

Sex is inferred only when reliable evidence exists. Ambiguous cases remain unresolved until explicit or structural information becomes available.

20260802-tree-function-design.md

Lineage Tree Function Design for biolineage

This document describes the technical design of the lineage tree rendering functions used by biolineage. It explains ancestor traversal, descendant traversal, cycle detection, ordering rules, and how the API returns tree structures to the

frontend. These functions power the `/entity/:id/entity-tree` endpoint and form the computational core of lineage visualization.

1. Overview

The lineage tree functions compute an entity tree for any individual in the database. An entity tree consists of:

- ancestors
- descendants
- spouses and partners
- siblings (derived from shared parents)
- generation ordering
- cycle-safe traversal

The tree is computed using PostgreSQL recursive queries and returned as a structured JSON object through the API.

The design is graph-first: entities are nodes, relationships are edges. The tree functions traverse these edges to build a complete lineage structure.

2. Ancestor Traversal

Ancestor traversal begins at a target entity and walks upward through parent edges. The traversal uses a recursive CTE with the following rules:

1. Start with the target entity.
2. Select all parents of the current generation.
3. Add each parent to the ancestor list.
4. Continue recursively until no further parents exist.
5. Track visited nodes to prevent cycles.
6. Record generation depth for ordering.

Generation depth is defined as:

- depth 0: the target entity
- depth 1: parents
- depth 2: grandparents
- depth 3: great-grandparents
- and so on

Ancestor traversal produces a list of nodes with their generation depth and relationship context.

3. Descendant Traversal

Descendant traversal mirrors ancestor traversal but walks downward through child edges.

Rules:

1. Start with the target entity.
2. Select all children of the current generation.
3. Add each child to the descendant list.
4. Continue recursively until no further children exist.
5. Track visited nodes to prevent cycles.
6. Record generation depth for ordering.

Generation depth for descendants is defined as:

- depth 0: the target entity
- depth 1: children
- depth 2: grandchildren
- depth 3: great-grandchildren
- and so on

Descendant traversal produces a list of nodes with their generation depth and relationship context.

4. Cycle Detection

Historical datasets often contain cycles, such as:

- repeated parentage
- incorrect relationships
- blended families with circular edges
- data entry errors

Cycle detection is implemented by maintaining a visited set inside the recursive CTE. When a node is encountered that has already been visited:

- the traversal stops for that branch
- the node is not added again
- the cycle is recorded internally for debugging

This ensures that lineage trees do not contain infinite loops or duplicate nodes.

5. Ordering Rules

Ordering is essential for rendering a readable lineage tree. The ordering rules are:

1. Sort ancestors by generation depth ascending.
2. Sort descendants by generation depth ascending.
3. Within each generation, sort by uuid for deterministic output.
4. Spouses and partners are listed alongside the target entity.
5. Siblings are derived from shared parents and grouped together.
6. Nodes with missing sex or ambiguous relationships are included but not used for ordering beyond depth.

These rules ensure stable output across ingestion runs and consistent rendering in the frontend.

6. Spouse and Partner Handling

Spouses and partners are not part of ancestor or descendant traversal. They are attached to the target entity as relational context.

Rules:

1. Retrieve all spouse or partner edges for the target entity.
2. Include spouses in the entity tree output.
3. Do not treat spouses as ancestors or descendants.
4. Use spouse relationships to infer sibling groups when children exist.

This ensures correct representation of blended families.

7. Sibling Derivation

Siblings are not stored directly in the database. They are derived from shared parents.

Rules:

1. Retrieve all parents of the target entity.
2. Retrieve all children of those parents.
3. Exclude the target entity.
4. Group siblings by shared parent sets.

Sibling derivation is performed after ancestor traversal and before descendant traversal.

8. API Output Structure

The `/entity/:id/entity-tree` endpoint returns a structured JSON object containing:

- entity
- ancestors
- descendants
- spouses
- siblings
- metadata

Example structure:

```
{
  "entity": { ... },
  "ancestors": [
    { "uuid": "...", "depth": 1, "relationship": "parent" },
    { "uuid": "...", "depth": 2, "relationship": "grandparent" }
  ],
  "descendants": [
    { "uuid": "...", "depth": 1, "relationship": "child" },
    { "uuid": "...", "depth": 2, "relationship": "grandchild" }
  ],
  "spouses": [
    { "uuid": "...", "relationship": "spouse" }
  ],
  "siblings": [
    { "uuid": "...", "relationship": "sibling" }
  ],
  "meta": {
    "cycles_detected": false,
    "generated_at": "timestamp"
  }
}
```

The API does not perform rendering. Rendering is handled by the frontend.

9. Performance Considerations

PostgreSQL recursive CTEs provide stable performance even for deep lineage trees. Performance optimizations include:

- indexing relationship edges
- indexing entity UUIDs
- caching normalized place lookups
- limiting traversal to relevant relationship types
- using deterministic ordering

The tree functions are designed to scale to thousands of nodes.

10. Summary

The lineage tree functions in biolineage are designed to be:

- graph-first
- cycle-safe
- deterministic
- deeply recursive
- structurally accurate
- consistent across ingestion runs

They form the computational foundation of the entity tree endpoint and support all lineage visualization features.

20260802-api-integration.md

API Integration for biolineage

This document describes the API endpoints exposed by biolineage. It explains request and response formats, Fastify route structure, serialization rules, how the API integrates with the lineage tree renderer, and how the API will integrate with future frontend components. This document serves as the reference for all external integrations.

1. Overview

The biolineage API is a lightweight, JSON-based interface built on Fastify. It exposes endpoints for:

- retrieving entity records
- retrieving events
- retrieving relationships
- retrieving normalized places
- computing lineage trees
- supporting future frontend components

The API is designed to be:

- predictable
- stable
- deterministic
- easy to consume
- compatible with modern frontend frameworks

All endpoints return structured JSON with consistent serialization rules.

2. Fastify Route Structure

Routes follow a simple pattern:

- `/entity/:uuid`
- `/entity/:uuid/events`
- `/entity/:uuid/relationships`
- `/entity/:uuid/entity-tree`
- `/place/:uuid`
- `/search/entity`
- `/search/place`

Each route is defined in a dedicated Fastify plugin. Plugins are loaded during server initialization to keep the route structure modular and maintainable.

Example route definition:

```
app.get('/entity/:uuid', async (req, reply) => {  
  const entity = await db.getEntity(req.params.uuid)  
  reply.send(entity)  
})
```

All routes return JSON.

3. Serialization Rules

Serialization is handled by Fastify's built-in JSON serializer. The rules are:

1. All UUIDs are returned as lowercase strings.
2. Dates are returned in ISO-8601 format when present.
3. Null values are preserved.
4. Arrays are always returned, even when empty.
5. Objects follow a stable key ordering for readability.
6. No additional metadata is added unless explicitly required.

Serialization is deterministic across all endpoints.

4. Entity Endpoints

4.1 `/entity/:uuid`

Returns a single entity record.

Response fields include:

- `uuid`
- `given_name`
- `family_name`

- sex
- notes
- created_at
- updated_at

4.2 /entity/:uuid/events

Returns all events associated with the entity.

Each event includes:

- uuid
- event_type
- date
- place_uuid
- description
- source

4.3 /entity/:uuid/relationships

Returns all relationship edges involving the entity.

Each relationship includes:

- uuid
- entity_a_uuid
- entity_b_uuid
- relationship_type
- inferred
- notes

5. Place Endpoints

5.1 /place/:uuid

Returns a normalized place record.

Fields include:

- uuid
- name
- country
- region
- city
- type
- notes

5.2 /search/place

Supports searching for places by name, region, or country.

6. Lineage Tree Endpoint

6.1 /entity/:uuid/entity-tree

This endpoint computes and returns the lineage tree for the target entity. It integrates directly with the tree rendering functions described in 20260802-tree-function-design.md.

The response includes:

- entity
- ancestors
- descendants
- spouses
- siblings
- meta

Example response structure:

```
{
  "entity": { ... },
  "ancestors": [
    { "uuid": "...", "depth": 1, "relationship": "parent" }
  ],
  "descendants": [
    { "uuid": "...", "depth": 1, "relationship": "child" }
  ],
  "spouses": [
    { "uuid": "...", "relationship": "spouse" }
  ],
  "siblings": [
    { "uuid": "...", "relationship": "sibling" }
  ],
  "meta": {
    "cycles_detected": false,
    "generated_at": "timestamp"
  }
}
```

The endpoint does not perform rendering. Rendering is handled entirely by the frontend.

7. Integration with the Lineage Tree Renderer

The lineage tree renderer is a backend subsystem that:

- performs ancestor traversal
- performs descendant traversal
- detects cycles
- derives siblings
- attaches spouses
- orders nodes by generation depth

The API simply calls the renderer and returns its output.

The renderer is designed to be:

- stateless
- deterministic
- fast
- safe for deep recursion

This ensures consistent results across all API calls.

8. Integration with Future Frontend Components

The API is designed to support future frontend features, including:

- interactive lineage trees
- entity detail pages
- event timelines
- place detail pages
- search interfaces
- relationship visualizations
- historical map overlays

The JSON structures returned by the API are intentionally simple and predictable to make frontend integration straightforward.

9. Error Handling

All endpoints follow consistent error handling rules:

- 404 for missing UUIDs
- 400 for invalid parameters
- 500 for unexpected server errors
- JSON error responses with a stable structure

Example error response:

```
{  
  "error": "Not Found",  
  "message": "Entity not found",  
  "statusCode": 404  
}
```

10. Summary

The biolineage API is designed to be:

- simple
- predictable
- stable
- easy to integrate
- compatible with modern frontend frameworks
- tightly coupled to the lineage tree renderer
- extensible for future features

It provides a clean interface for retrieving entities, events, relationships, places, and lineage trees.

20260802-sqlite-to-postgresql-tree-comparison.md

Comparison of Lineage Tree Computation: SQLite vs PostgreSQL

This document compares the original lineage tree logic implemented in SQLite with the redesigned PostgreSQL implementation used by **biolineage**. It explains the differences in recursion strategy, improvements in ancestor/descendant traversal, correctness issues discovered in the SQLite version, and the validation tests used to confirm the correctness of the PostgreSQL model.

This comparison demonstrates why PostgreSQL is the correct long-term home for lineage computation.

1. Background

The original norm.app lineage tree was computed using SQLite recursive queries. While functional for small datasets, SQLite's recursion model revealed several limitations when applied to a large, multi-generational dataset with inferred relationships, ambiguous parentage, and historical inconsistencies.

The PostgreSQL redesign addresses these limitations by providing:

- deeper recursion

- cycle detection
- stable ordering
- correct ancestor/descendant traversal
- predictable performance
- a graph-first relationship model

This document outlines the differences.

2. Differences in Recursion Strategy

SQLite Recursion Model

SQLite supports recursive CTEs, but with constraints:

- recursion depth is limited by default
- cycle detection must be implemented manually
- ordering inside recursive queries is unstable
- performance degrades significantly with large graphs
- recursive queries cannot easily incorporate multiple relationship types

The original implementation relied on:

- a single recursive CTE
- manual parent/child traversal
- no cycle detection
- no inferred relationship support

This worked for simple trees but broke down with complex family structures.

PostgreSQL Recursion Model

PostgreSQL provides a more robust recursive CTE engine:

- deeper recursion without performance collapse
- stable ordering within recursive layers
- ability to join multiple tables inside recursion
- better cycle detection patterns
- support for multi-edge graph traversal
- predictable performance even with thousands of nodes

The redesigned lineage tree uses:

- recursive CTEs for ancestor traversal
- recursive CTEs for descendant traversal
- cycle detection via visited-set accumulation
- stable ordering using generation depth
- relationship-type filtering

- inferred edges included when appropriate

This results in correct, complete lineage trees.

3. Improvements in Ancestor/Descendant Traversal

SQLite Limitations

The SQLite version exhibited several traversal issues:

- **Missing ancestors** when recursion depth was exceeded
- **Duplicate ancestors** when cycles were present
- **Incorrect ordering** (children appearing before parents)
- **Spouse relationships incorrectly treated as ancestors**
- **Inferred relationships ignored**
- **Ambiguous parentage causing traversal gaps**

These issues were visible in the entity tree output and required manual correction.

PostgreSQL Improvements

The PostgreSQL implementation resolves all of the above:

- **Full ancestor traversal** regardless of depth
- **Cycle detection** prevents infinite loops
- **Stable ordering** based on generation level
- **Correct relationship filtering** (parent/child only)
- **Inferred relationships included** when flagged
- **Ambiguous parentage handled deterministically**
- **Descendant traversal matches ancestor traversal symmetry**

The result is a lineage tree that is:

- complete
- correct
- stable
- reproducible

This correctness is essential for downstream features such as gender inference and event aggregation.

4. Correctness Issues Found in the SQLite Version

During the migration, several correctness issues were discovered in the SQLite implementation:

A. Missing ancestors

Certain individuals lacked grandparents or great-grandparents due to recursion depth limits.

B. Incorrect spouse traversal

Spouses were sometimes treated as ancestors or descendants due to relationship misclassification.

C. Cycles causing duplication

Cycles in the relationship graph (common in historical datasets) caused repeated nodes.

D. Ambiguous parentage

Individuals with multiple possible parents were inconsistently resolved.

E. Ordering instability

The same query produced different ordering depending on dataset state.

F. No inferred relationships

Relationships inferred from events (e.g., birth event $\rightarrow$ parent linkage) were not included.

G. Performance degradation

Large trees caused slow queries or timeouts.

All of these issues are resolved in the PostgreSQL version.

5. Validation Tests Comparing Both Outputs

To ensure correctness, a series of validation tests were run comparing:

- SQLite person tree output
- PostgreSQL entity tree output

These tests included:

1. Direct comparison of ancestor lists

Every ancestor in SQLite was checked against PostgreSQL output.

2. Depth comparison

PostgreSQL consistently produced deeper trees.

3. Cycle detection tests

PostgreSQL avoided duplicate nodes; SQLite did not.

4. Relationship filtering tests

PostgreSQL correctly excluded spouses and non-parental relationships.

5. Inferred relationship tests

PostgreSQL included inferred parents; SQLite did not.

6. Performance tests

PostgreSQL completed deep trees in milliseconds; SQLite slowed significantly.

7. Event-linked traversal tests

PostgreSQL correctly linked events to places and entities during traversal.

The PostgreSQL implementation passed all validation tests.

6. Why PostgreSQL Is the Correct Long-Term Home

PostgreSQL is the correct long-term home for lineage computation because it provides:

- **robust recursion**
- **stable ordering**
- **cycle detection**
- **graph-friendly relationship modeling**
- **event-centric historical modeling**
- **place normalization integration**
- **scalability for large datasets**
- **predictable performance**

SQLite served as a useful prototype, but PostgreSQL is required for correctness, stability, and future expansion.

appendix.md

Project Appendix for biolineage

This appendix contains project-level information not covered in the technical documents. It includes ownership, attribution, copyright assignment, AI involvement, geographical data sources, the tandem geography project, static site information, and long-term hosting commitments.

1. Project Ownership and Attribution

biolineage is a saiSquared project. All copyright for the codebase, documentation, and associated assets is assigned to saiSquared.

Credit for the creation, design, and development of the project is shared among:

- Norman Eaddy
- Charles Carroll
- Chris Rowley

These individuals contributed the historical data, design direction, engineering work, and architectural decisions that form the foundation of the project.

2. AI Involvement in Development

biolineage is not a vibe-coded product. While portions of the codebase and documentation were AI-generated, all generated material was:

- reviewed
- tested
- corrected
- iterated upon
- integrated intentionally into the architecture

Microsoft Copilot acted as a team member during development, providing architectural reasoning, code generation, normalization logic, and documentation support. All final implementation decisions were made by the human developers listed above.

3. Geographical Data Sources

The project uses geographical data from many sources, including:

- historical atlases
- ISO-3166 datasets
- TLD registries
- county and parish records
- cemetery and locality databases

- sovereign entity reference materials
- historical maps
- manually curated corrections
- narrative descriptions from Norm's dataset

This geographical dataset is incomplete and will continue to evolve. Many localities require human review, and ambiguous cases are documented in places-issues.md.

4. Tandem Geography Project

A tandem project named geography is responsible for maintaining structured geopolitical data. This project defines sovereign entities, subdivisions, administrative divisions, and municipalities. These tables provide the authoritative reference for countries, states, provinces, counties, parishes, and cities.

The PostgreSQL schema for the geography project includes the following tables:

sovereign_entities

```
sovereign_entities: {
  id: UUID,
  name: TEXT,
  long_name: TEXT,
  type: TEXT,
  iso31661: JSONB,
  has_flag: BOOLEAN,
  flag_file: TEXT,
  has_armorial: BOOLEAN,
  armorial_type: TEXT,
  armorial_file: TEXT,
  tlds: JSONB
}
```

subdivisions

```
subdivisions: {
  id: UUID,
  sovereign_entity_id: UUID,
  name: TEXT,
  type: TEXT,
  iso31662: JSONB,
  has_flag: BOOLEAN,
  flag_file: TEXT,
  has_armorial: BOOLEAN,
  armorial_type: TEXT,
}
```

```

    armorial_file: TEXT
}

administrative_divisions
administrative_divisions: {
    id: UUID,
    sovereign_entity_id: UUID,
    subdivision_id: UUID,
    name: TEXT,
    long_name: TEXT,
    type: TEXT,
    fips: INTEGER,
    latitude: DOUBLE PRECISION,
    longitude: DOUBLE PRECISION,
    iso31662: JSONB,
    meta: JSONB,
    has_flag: BOOLEAN,
    flag_file: TEXT,
    has_armorial: BOOLEAN,
    armorial_type: TEXT,
    armorial_file: TEXT
}

```

municipalities

```

municipalities: {
    id: UUID,
    sovereign_entity_id: UUID,
    subdivision_id: UUID,
    administrative_division_id: UUID,
    name: TEXT,
    long_name: TEXT,
    type: TEXT,
    latitude: DOUBLE PRECISION,
    longitude: DOUBLE PRECISION,
    meta: JSONB,
    has_flag: BOOLEAN,
    flag_file: TEXT,
    has_armorial: BOOLEAN,
    armorial_type: TEXT,
    armorial_file: TEXT
}

```

These tables support:

- ISO-3166-1 and ISO-3166-2 codes

- historical and modern sovereign entities
- flags and armorial bearings
- TLDs
- FIPS codes
- latitude/longitude coordinates
- metadata for future expansion

The geography project is designed to be a long-term, authoritative reference for all geopolitical data used by biolineage.

5. Static Site Files

The `sites/` folder contains static files served directly by nginx. These files include:

- static HTML
- CSS
- JavaScript bundles
- images
- assets for the frontend

The backend does not process or transform files in `sites/`. nginx serves them as-is.

6. Long-Term Hosting Commitment

An instance of biolineage will be hosted and paid for by saiSquared in perpetuity for Norman Eaddy and his descendants. This commitment ensures that the lineage data, historical records, and associated research remain accessible to future generations of the Eaddy family.

The hosted instance will:

- run the same codebase as the public version
- receive updates and improvements
- preserve all historical data ingested into the system
- remain available without subscription or usage fees

This long-term hosting guarantee is a core part of the project's mission.

7. Summary

This appendix provides project-level information that supplements the technical documentation. It defines ownership, attribution, AI involvement, geographical data sources, the tandem geography project, static site information, and the

long-term hosting commitment. These details complete the documentation set for biolineage.

Places Issues

Fixes

```
const placeFixes = [
  {
    ogName: 'Marion County',
    ogCountry: null,
    ogRegion: null,
    ogCity: null,
    uuid: 'b8123bc6-fb68-475d-96b6-a8634ef87524'
  },
  {
    ogName: 'Marengo Coounty',
    ogCountry: 'us',
    ogRegion: 'AL',
    ogCity: null,
    uuid: '02be28fd-6498-478a-b3e2-ebf37029fa75'
  },
  {
    ogName: 'Shands Hospital',
    ogCountry: 'us',
    ogRegion: 'FL',
    ogCity: 'Gainesville',
    uuid: 'bf35f0e7-f801-452b-930e-e2d7fd47d1c0'
  },
  {
    ogName: 'Sumter Cemetary',
    ogCountry: 'us',
    ogRegion: 'SC',
    ogCity: 'Sumter',
    uuid: 'a0a35d5b-6fbe-4160-aaba-843a4dc8f1a5'
  },
  {
    ogName: 'Old Johnsonville Cemetary',
    ogCountry: 'us',
    ogRegion: 'SC',
    ogCity: 'Johnsonville',
    uuid: '65a31b18-d052-475b-873a-34fd1393457a'
  },
  {
    ogName: 'Old Johnsonville Cemetary',
```

```

    ogCountry: 'us',
    ogRegion: 'SC',
    ogCity: 'Jonhsonville',
    uuid: '65a31b18-d052-475b-873a-34fd1393457a'
  },
  {
    ogName: null,
    ogCountry: 'de',
    ogRegion: 'BW',
    ogCity: 'Leinfelden Stuttgart',
    uuid: '201c8ccb-8124-4e4c-bda4-60757ef86d40'
  },
  {
    ogName: null,
    ogCountry: 'us',
    ogRegion: null,
    ogCity: 's',
    uuid: '5a870dc8-aff3-49d3-8ef7-47099457264e'
  },
  {
    ogName: null,
    ogCountry: 'us',
    ogRegion: 'MD',
    ogCity: 'Silver Sprinig',
    uuid: '712e4e62-29e3-4090-bf3e-314a3695a1b0'
  },
  {
    ogName: null,
    ogCountry: 'us',
    ogRegion: 'MD',
    ogCity: 'Silver spring',
    uuid: '712e4e62-29e3-4090-bf3e-314a3695a1b0'
  },
  {
    ogName: null,
    ogCountry: 'us',
    ogRegion: 'NY',
    ogCity: 'NYC',
    uuid: 'e7e00cba-707f-435c-87b1-9593f5f04392'
  },
  {
    ogName: null,
    ogCountry: 'us',
    ogRegion: 'NY',
    ogCity: 'New York',
    uuid: 'e7e00cba-707f-435c-87b1-9593f5f04392'
  }

```

```

},
{
  ogName: null,
  ogCountry: 'us',
  ogRegion: 'SC',
  ogCity: 'Lynches Rover',
  uuid: '5d3a8488-37c5-469a-95b7-85b7362a4f15'
},
{
  ogName: null,
  ogCountry: 'us',
  ogRegion: 'SC',
  ogCity: 'Sumte',
  uuid: '669063ea-d9c5-4faa-9b0b-6267b9bc62bb'
},
{
  ogName: '?',
  ogCountry: 'us',
  ogRegion: 'SC',
  ogCity: 'Sumter',
  uuid: '669063ea-d9c5-4faa-9b0b-6267b9bc62bb'
},
{
  ogName: 'Evergreen Cemetary',
  ogCountry: 'us',
  ogRegion: 'SC',
  ogCity: 'Sumter',
  uuid: '4de025c7-059b-446b-996c-7b8cae18f9df'
},
{
  ogName: 'tbd',
  ogCountry: 'us',
  ogRegion: null,
  ogCity: null,
  uuid: '5a870dc8-aff3-49d3-8ef7-47099457264e'
},
{
  ogName: 'Home',
  ogCountry: 'us',
  ogRegion: 'SC',
  ogCity: 'Sumter',
  uuid: '669063ea-d9c5-4faa-9b0b-6267b9bc62bb'
},
{
  ogName: 'Manning Cemetery age 57.5 hours',
  ogCountry: 'us',

```



```

    ogRegion: 'SC',
    ogCity: 'Manning',
    uuid: '3afff307-6683-4d7b-8fd3-4cbebb8b5aee'
  },
  {
    ogName: 'Manning Cemetrery',
    ogCountry: 'us',
    ogRegion: 'SC',
    ogCity: 'Manning',
    uuid: '3afff307-6683-4d7b-8fd3-4cbebb8b5aee'
  },
  {
    ogName: 'some say born 1732',
    ogCountry: 'gb',
    ogRegion: null,
    ogCity: null,
    uuid: 'd3c13577-3379-47e7-b9f1-7c55d4542f21'
  },
  {
    ogName: 'Robeson County, NC',
    ogCountry: 'us',
    ogRegion: 'NC',
    ogCity: null,
    uuid: 'a8132416-8bd8-4feb-b7c1-4914101e30c7'
  },
  {
    ogName: 'a',
    ogCountry: 'us',
    ogRegion: 'CA',
    ogCity: null,
    uuid: '5b98ae04-9502-40d7-ad6a-a8eb00bc8b9f'
  },
  {
    ogName: 'died young',
    ogCountry: 'us',
    ogRegion: 'SC',
    ogCity: 'Timmons ville',
    uuid: '45768b17-ee48-4a9e-a7e8-4c9d0e4745cb'
  },
  {
    ogName: 'some say died 9/30/1819',
    ogCountry: 'us',
    ogRegion: 'SC',
    ogCity: 'Lynches River',
    uuid: '5d3a8488-37c5-469a-95b7-85b7362a4f15'
  },
},

```

```

    {
      ogName: 'Tappan',
      ogCountry: null,
      ogRegion: 'NY',
      ogCity: null,
      uuid: '5daaa0a7-aeb3-4e0f-97d0-4ef8150b54fb'
    }
  ]

  const fixPlace = (data) => {
    const match = placeFixes.find(rule =>
      rule.ogName === data.ogName &&
      rule.ogCountry === data.ogCountry &&
      rule.ogRegion === data.ogRegion &&
      rule.ogCity === data.ogCity
    )

    return match ? match.uuid : data.uuid
  }

```

Discussion

These require decision from Norm and Charles

Case: New York vs Orange (New York)

- **id 1:** “New York”, no city
- **id 2:** “Orange (New York)”, city = Orange

Issue:

- “New York” is ambiguous (state? city? county?).
- “Orange” may refer to Orange County or the town of Orange.
- Structured fields do not clarify intended level.

Decision needed:

- Should “New York” be treated as the state or the city?
 - Should “Orange” be normalized to Orange County?
 - Are these separate intended places?
-

Case: Texas vs Corsicana (Texas)

- **id 3:** “Texas”, no city
- **id 4:** “Corsicana (Texas)”, city = Corsicana

Issue:

- “Texas” is ambiguous (state vs general location).
- “Corsicana” is a specific city.
- Unclear whether “Texas” is meant as a general birthplace or missing city.

Decision needed:

- Should “Texas” be treated as the state?
 - Should “Texas” be merged with Corsicana if referring to same entity?
 - Or kept separate?
-

Case: Leinfelden Stuttgart vs Musberg Stuttgart (Germany)

- **id 8:** “Leinfelden Stuttgart”
- **id 9:** “Musberg Stuttgart”

Issue:

- Both are concatenations of Stuttgart + a district of Leinfelden-Echterdingen.
- Not actual city names.
- Unclear intended locality.

Decision needed:

- Should these be split into Stuttgart vs Leinfelden-Echterdingen?
 - Should they be merged?
 - Should they be rewritten as districts?
-

Case: South Korea ambiguous localities

- **id 14:** Hamongji
- **id 15:** Hamung
- **id 16:** Jungpyongkun
- **id 17:** Poyngyang
- **id 19:** Hamung (HKN)

Issue:

- All appear to be mis-transliterations.
- “Poyngyang” is Pyongyang (North Korea).
- “Hamung” likely Hamhung (North Korea).
- Country field incorrectly says South Korea.

Decision needed:

- Confirm intended country (North vs South Korea).
- Confirm intended locality.

- Decide whether to merge Hamung variants.
-

Case: Overijssel (Netherlands)

- **id 23:** Overijssel (Netherlands)

Issue:

- Overijssel is a province, not a city.
- Structured fields treat it as a city.

Decision needed:

- Should this be converted to Region?
 - Or kept as Discussion?
-

Case: Silver Valley (United States)

- **id 29:** Silver Valley (United States)

Issue:

- Multiple Silver Valleys exist (NC, ID, CA).
- No region or city provided.

Decision needed:

- Identify intended state.
 - Decide whether to create multiple Silver Valley entries.
-

Case: Escambia vs Escambria (Florida)

- **id 42:** Escambia
- **id 43:** Escambria

Issue:

- Escambia is a county, not a city.
- Escambria is a misspelling.
- Structured fields treat both as cities.

Decision needed:

- Should both be normalized to Escambia County?
 - Should they remain separate?
 - Should city be removed?
-

Case: Franklin (Florida)

- **id 44:** Franklin (Florida)

Issue:

- Franklin is a county, not a city.
- Structured fields treat it as a city.

Decision needed:

- Should this be converted to County?
 - Or kept as ambiguous?
-

Case: Tyrrell County ambiguity

- **id 62:** Tyrrell (North Carolina)

Issue:

- Tyrrell is a county, not a city.
- Structured fields treat it as a city.

Decision needed:

- Should this be converted to County?
 - Should city be removed?
-

Case: Bullington (New Jersey)

- **id 63:** Bullington (New Jersey)

Issue:

- No known locality named Bullington in NJ.
- Likely mis-entry.

Decision needed:

- Identify intended place.
 - Possibly remove or rewrite.
-

Case: Camden (New Jersey)

- **id 64:** Camden (New Jersey)

Issue:

- Misspelling of Camden.

- Needs confirmation.

Decision needed:

- Should this be corrected to Camden?
 - Or kept separate?
-

Case: Orangetown vs Tappan (New York)

- **id 74:** Orangetown
- **id 75:** Tappan

Issue:

- Both are hamlets/towns in Rockland County.
- Structured fields treat them as cities.
- “Bew York” misspelling corrected.

Decision needed:

- Should these be typed as Town or Community?
 - Should they be linked to Rockland County?
-

Case: Anderson County Hospital vs Anderson County Cemetery

- **id 105:** Anderson County Hospital
- **id 106:** Anderson County Cemetery

Issue:

- Both refer to Anderson County but different facility types.
- Cemetery is in Pendleton; hospital location unclear.

Decision needed:

- Confirm hospital location.
 - Confirm cemetery location.
 - Decide whether to create Anderson County as a region.
-

Case: Arlington National Cemetery (Springfield, VA)

- **id 107:** Arlington National Cemetery (Springfield)

Issue:

- Arlington National Cemetery is in Arlington, not Springfield.

- Structured fields incorrect.

Decision needed:

- Correct city?
 - Keep as ambiguous?
-

Case: Berkeley/Berkley County variants

- **id 111:** Berkeley County
- **id 112:** Berkley County (Christ Church Parish)
- **id 113:** Berkley County (Saewee Bay)

Issue:

- Spelling variants.
- Multiple associated localities.
- Some historic (Christ Church Parish).

Decision needed:

- Should all be merged to Berkeley County?
 - Should historic parish variants remain separate?
-

Case: Johnsonville (Columbia)

- **id 116:** Johnsonville (Columbia)

Issue:

- “Buried at Johnsonville SC” placed in city field.
- Columbia is incorrect.

Decision needed:

- Should city be removed?
 - Should this be merged with Johnsonville?
-

Case: Denton Town Cemetery (Lansdowne, MD)

- **id 131:** Denton Town Cemetery (Lansdowne)

Issue:

- Denton is in Caroline County, not Lansdowne.
- Structured fields incorrect.

Decision needed:

- Confirm intended cemetery.
 - Correct locality.
-

Case: Dixon Mills (Marengo County)

- **id 132:** Dixon Mills?

Issue:

- “Dixon Mills?” indicates uncertainty.
- Needs confirmation.

Decision needed:

- Confirm intended locality.
 - Remove question mark.
-

Case: Dorcetshire (LymeRegis)

- **id 133:** Dorcetshire (LymeRegis)

Issue:

- Misspelling of Dorsetshire.
- Historic county name.
- Mixed with city.

Decision needed:

- Should this be normalized to Dorset?
 - Should historic name be preserved?
-

Case: Gainesville (Florida)

- **id 138:** Gainesville FL?

Issue:

- “FL?” indicates uncertainty.
- Missing structured fields.

Decision needed:

- Confirm intended locality.
 - Remove question mark.
-

Case: Gates of Heaven Cemetery (Maryland)

- **id 139:** Gates of Heaven Cemetery

Issue:

- Missing city (Aspen Hill).
- Needs confirmation.

Decision needed:

- Should city be added?
 - Should this be normalized?
-

Case: Ladies Clinic (Japan)

- **id 153:** Ladies Clinic (Japan)

Issue:

- Generic name.
- No city or region.
- Cannot normalize.

Decision needed:

- Identify intended locality.
 - Possibly remove.
-

Case: Lexington (Columbia)

- **id 154:** Lexington (Columbia)

Issue:

- Lexington is a city in Lexington County.
- Columbia is incorrect.

Decision needed:

- Should city be removed?
 - Should this be merged with Lexington?
-

Case: Louisville Bluegrass Planting Grounds (Netherlands)

- **id 155:** Louisville Bluegrass Planting Grounds

Issue:

- Kentucky horse facility placed in Netherlands.
- “Horse-Raising Town” is narrative.

Decision needed:

- Identify intended locality.
 - Remove narrative text.
-

Case: New Hope Burying Ground variants

- **id 163:** Darlington District
- **id 164:** Manning

Issue:

- Same cemetery name, different associated places.
- One historic district, one modern city.
- Narrative text in ogName.

Decision needed:

- Confirm actual cemetery location.
 - Decide whether to merge or keep separate.
-

Case: Old Johnsonville Cemetery variants

- **id 168:** Old Johnsonville Cemetery
- **id 169:** Old Johnsonville Methodist Church Cemetery
- **id 170:** Ole Johnsonville Cemetery

Issue:

- Multiple naming variants.
- Denominational variant.
- Misspellings (“Ole”, “Cemetary”).
- Likely same cemetery or adjacent ones.

Decision needed:

- Confirm whether these are one or multiple cemeteries.
- Decide whether to merge.

20260802-oidc-sso.md

Proposal: Migration from Email/Password Authentication to OIDC SSO

This document proposes replacing the current private email/password authentication system with an OpenID Connect (OIDC) Single Sign-On (SSO) model. It

describes the limitations of the existing approach, the benefits of OIDC, expected architectural changes, and the migration plan for biolineage.

1. Overview

biolineage currently uses a private email/password login mechanism. This system is functional but limited, and does not align with modern authentication standards. OIDC SSO provides a secure, standardized, and maintainable alternative.

This proposal outlines the intent to migrate to OIDC SSO in a future milestone. It does not represent completed work.

2. Limitations of the Current Authentication Model

The existing email/password system has several constraints:

- passwords must be stored, hashed, and managed
- password resets require manual handling
- no multi-factor authentication
- no federated identity support
- no session revocation or centralized identity control
- limited auditability
- not suitable for future multi-instance deployments

While acceptable for early development, it is not ideal for long-term operation.

3. Benefits of OIDC SSO

Migrating to OIDC SSO provides:

- secure, standards-based authentication
- no password storage in biolineage
- support for multi-factor authentication
- centralized identity management
- compatibility with Azure AD, Auth0, Keycloak, and other providers
- easier user provisioning and deprovisioning
- improved audit logging
- future compatibility with additional frontend components
- simplified session handling via Redis

OIDC is widely supported and aligns with modern best practices.

4. Candidate Identity Providers

Several OIDC providers are under consideration:

- Azure Active Directory
- Auth0
- Keycloak
- Okta
- self-hosted OIDC providers

The final choice will depend on cost, ease of integration, and long-term maintainability.

5. Expected Backend Changes

Migrating to OIDC will require updates to the backend:

- add OIDC client configuration to the Fastify server
- replace email/password login routes with OIDC redirect endpoints
- validate ID tokens and user claims
- store session identifiers in Redis
- remove password hashing and storage logic
- update user table to store OIDC subject identifiers
- add logout and session revocation endpoints

These changes will be implemented in a future milestone.

6. Expected Frontend Changes

Frontend authentication will shift from:

- manual login forms
- password fields
- local session cookies

to:

- OIDC login redirects
- provider-managed authentication screens
- standardized callback handling
- token-based session initialization

The frontend will not store passwords or handle password resets.

7. Session Storage

Redis will serve as the session store for OIDC:

- stores session tokens
- supports expiration and revocation
- integrates cleanly with Fastify
- avoids storing session data in PostgreSQL

This improves performance and simplifies authentication logic.

8. Migration Plan

The migration will occur in phases:

1. introduce OIDC client configuration
2. add login and callback routes
3. implement Redis-backed session storage
4. migrate user records to OIDC subject identifiers
5. deprecate email/password login
6. remove password storage and hashing logic
7. update frontend to use OIDC redirects
8. finalize documentation and update API references

The existing login system will remain functional until the migration is complete.

9. Backward Compatibility

During migration:

- both authentication systems may temporarily coexist
- existing users will be mapped to OIDC identities
- no historical data will be affected
- API behavior will remain unchanged

Once OIDC is fully deployed, the email/password system will be removed.

10. Summary

This proposal describes the planned migration from private email/password authentication to OIDC SSO. The change will improve security, maintainability, and long-term viability. Implementation will occur in a future milestone and will not affect existing lineage, event, or place data.